

Tree 2

Q.No:1

Question description

Given two rooted trees, your task is to find out if they are *isomorphic*, i.e., it is possible to draw them so that they look the same.

Constraints

- $1 \leq t \leq 1000$
- $2 \leq n \leq 10^5$
- the sum of all values of n is at most 10^5

Input

The first input line has an integer t : the number of tests. Then, there are t tests described as follows:

The first line has an integer n : the number of nodes in both trees. The nodes are numbered $1, 2, \dots, n$, and node 1 is the root.

Then, there are $n-1$ lines describing the edges of the first tree, and finally $n-1$ lines describing the edges of the second tree.

Output

For each test, print "YES", if the trees are isomorphic, and "NO" otherwise.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
2
3
1 2
2 3
1 2
1 3
3
1 2
2 3
1 3
3 2
```

Test Case 2

INPUT (STDIN)

```
2
3
1 2
2 3
1 3
3 3
3
1 2
2 3
1 3
3 3
```

EXPECTED OUTPUT

```
NO
YES
```

EXPECTED OUTPUT

```
NO
NO
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(t--)
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100005
#define MAXE (2 * MAXN)

int head1[MAXN], head2[MAXN];
int to1[MAXE], next1[MAXE];
int to2[MAXE], next2[MAXE];

int ec1, ec2;
```

```
void initGraph(int head[])
{
    for (int i = 1; i < MAXN; i++)
        head[i] = -1;
}

void addEdge1(int u, int v)
{
    to1[ec1] = v;
    next1[ec1] = head1[u];
    head1[u] = ec1++;
}

void addEdge2(int u, int v)
{
    to2[ec2] = v;
    next2[ec2] = head2[u];
    head2[u] = ec2++;
}

typedef struct
{
    unsigned long long h1;
    unsigned long long h2;
} Hash;

int cmpHash(const void *a, const void *b)
{
    Hash *x = (Hash *)a;
    Hash *y = (Hash *)b;

    if (x->h1 < y->h1) return -1;
    if (x->h1 > y->h1) return 1;

    if (x->h2 < y->h2) return -1;
    if (x->h2 > y->h2) return 1;

    return 0;
}

Hash buildTree(
    int n,
    int head[],
    int to[],
    int next[]
)
{
    int parent[MAXN];
    int order[MAXN];
    int stack[MAXN];

    Hash sub[MAXN];

    int top = 0;
```

```
int ord = 0;

for (int i = 1; i <= n; i++)
    parent[i] = 0;
stack[top++] = 1;
parent[1] = -1;
while (top > 0)
{
    int u = stack[--top];

    order[ord++] = u;

    for (int e = head[u]; e != -1; e = next[e])
    {
        int v = to[e];

        if (v == parent[u])
            continue;

        parent[v] = u;
        stack[top++] = v;
    }

    for (int i = ord - 1; i >= 0; i--)
    {
        int u = order[i];

        int childCount = 0;

        for (int e = head[u]; e != -1; e = next[e])
        {
            int v = to[e];

            if (parent[v] == u)
                childCount++;
        }

        Hash *children = NULL;

        if (childCount > 0)
        {
            children = (Hash *)malloc(
                childCount * sizeof(Hash)
            );

            int k = 0;

            for (int e = head[u]; e != -1; e = next[e])
            {
                int v = to[e];
```

```
if (parent[v] == u)
{
children[k++] = sub[v];
}
}
qsort(
children,
childCount,
sizeof(Hash),
cmpHash
);
}
unsigned long long h1 =
1469598103934665603ULL;

unsigned long long h2 =
1099511628211ULL;
h1 ^= (unsigned long long)(childCount + 1007);
h1 *= 1099511628211ULL;

h2 ^= (unsigned long long)(childCount + 2003);
h2 *= 1469598103934665603ULL;

for (int j = 0; j < childCount; j++)
{
h1 ^= children[j].h1 + 0x9e3779b97f4a7c15ULL;
h1 *= 1099511628211ULL;

h2 ^= children[j].h2 + 0xc2b2ae3d27d4eb4fULL;
h2 *= 0x100000001b3ULL;
}

sub[u].h1 = h1;
sub[u].h2 = h2;

if (children != NULL)
free(children);
}

return sub[1];
}
int main()
{
int t;

scanf("%d", &t);

while (t--)
{
int n;

scanf("%d", &n);
```

```
initGraph(head1);
initGraph(head2);

ec1 = 0;
ec2 = 0;

for (int i = 0; i < n - 1; i++)
{
    int u, v;

    scanf("%d %d", &u, &v);

    addEdge1(u, v);
    addEdge1(v, u);
}
for (int i = 0; i < n - 1; i++)
{
    int u, v;

    scanf("%d %d", &u, &v);

    addEdge2(u, v);
    addEdge2(v, u);
}
Hash tree1 = buildTree(
n,
head1,
to1,
next1
);

Hash tree2 = buildTree(
n,
head2,
to2,
next2
);
if (tree1.h1 == tree2.h1 &&
tree1.h2 == tree2.h2)
{
    printf("YES\n");
}
else
{
    printf("NO\n");
}

return 0;
}
```

Output:

Test_case:1

```
2
3
1 2
2 3
1 2
1 3
NO
3
1 2
2 3
1 3
3 2
YES
o aadityajaiswal@Aaditya
```

Test_case:2

```
o aadityajaiswal@Aaditya
2
3
1 2
2 3
1 3
3 3
NO
3
1 2
2 3
1 3
3 3
NO
o aadityajaiswal@Aaditya
```